```
# ============================================================================
# 3-STOREY MASONRY FRAME — DEBUG VERSION
# Same as the original script, with:
#   - catch blocks around every analysis phase
#   - verbose output at each step
#   - fixes applied for the three critical issues found in review
#     (settlement DOF, floor load formula, pushover fallback)
# ============================================================================

proc section {title} {
    puts "\n============================================================"
    puts "  $title"
    puts "============================================================"
}

proc check {label result} {
    if {$result == 0} {
        puts "  \[OK\]  $label"
    } else {
        puts "  \[FAIL\] $label  (return code: $result)"
    }
}

# ============================================================================
# SETUP
# ============================================================================

section "MODEL SETUP"

if {[catch {wipe} err]} { puts "  wipe: $err" }
if {[catch {model basic -ndm 3 -ndf 6} err]} {
    puts "  FATAL — model setup failed: $err"
    return
}
puts "  model basic -ndm 3 -ndf 6 : OK"

set H_story 3.0
set L_span  3.0
set L_pier  1.0
set T_pier  0.25
set H_span  0.80

set E       1700.0e6
set G       550.0e6
set fc      6.0e6
set c       0.150e6
set Gc      6.0
set mu0     0.40
set beta    0.30

set rho     1200.0
set g       9.81

# ============================================================================
# NODES
# ============================================================================

section "NODES"

if {[catch {

    node 1   0.0      0.0   0.0
    node 2   $L_span  0.0   0.0

    node 3   0.0      0.0   $H_story
    node 4   $L_span  0.0   $H_story

    node 5   0.0      0.0   [expr 2.0*$H_story]
    node 6   $L_span  0.0   [expr 2.0*$H_story]

    node 7   0.0      0.0   [expr 3.0*$H_story]
    node 8   $L_span  0.0   [expr 3.0*$H_story]
```

```
    node 9   0.0      0.0   [expr 0.5*$H_story]
    node 10  $L_span  0.0   [expr 0.5*$H_story]
    node 11  0.0      0.0   [expr 1.5*$H_story]
    node 12  $L_span  0.0   [expr 1.5*$H_story]
    node 13  0.0      0.0   [expr 2.5*$H_story]
    node 14  $L_span  0.0   [expr 2.5*$H_story]

    node 15  [expr $L_span/2.0] 0.0 $H_story
    node 16  [expr $L_span/2.0] 0.0 [expr 2.0*$H_story]
    node 17  [expr $L_span/2.0] 0.0 [expr 3.0*$H_story]

    puts "  All 17 nodes created : OK"

} err]} {
    puts "  FATAL — node creation failed: $err"
    return
}

# ============================================================================
# BOUNDARY CONDITIONS
# FIX #1 (from review): node 1 DOF 3 (vertical) left FREE for settlement
# ============================================================================

section "BOUNDARY CONDITIONS"

if {[catch {
    fix 1  1 1 0 1 1 1    ;# vertical DOF (3) free — needed for settlement
    fix 2  1 1 1 1 1 1
    puts "  node 1 : fixed in X Y Rx Ry Rz — vertical (Z) FREE for settlement"
    puts "  node 2 : fully fixed"
} err]} {
    puts "  FATAL — boundary conditions failed: $err"
    return
}

# ============================================================================
# ELEMENTS — PIERS
# ============================================================================

section "PIER ELEMENTS"

foreach {eid ni nj nk} {
    1   1  3  9
    2   2  4  10
    3   3  5  11
    4   4  6  12
    5   5  7  13
    6   6  8  14
} {
    if {[catch {
        element Macroelement3d $eid \
            $ni $nj $nk \
            0.0 0.0 1.0 \
            0.0 1.0 0.0 \
            -tremuri \
            $H_story $L_pier $T_pier \
            $E $G $fc $mu0 $c $Gc $beta \
            -density $rho \
            -cmass \
            -pDelta
        puts "  Pier element $eid (nodes $ni-$nj-$nk) : OK"
    } err]} {
        puts "  FAILED — pier element $eid: $err"
    }
}

# ============================================================================
# ELEMENTS — SPANDRELS
# ============================================================================

section "SPANDREL ELEMENTS"
```

```
foreach {eid ni nj nk} {
    7   3  4  15
    8   5  6  16
    9   7  8  17
} {
    if {[catch {
        element Macroelement3d $eid \
            $ni $nj $nk \
            1.0 0.0 0.0 \
            0.0 1.0 0.0 \
            -tremuri \
            $H_span $L_span $T_pier \
            $E $G $fc $mu0 $c $Gc $beta \
            -density $rho \
            -cmass \
            -pDelta
        puts "  Spandrel element $eid (nodes $ni-$nj-$nk) : OK"
    } err]} {
        puts "  FAILED — spandrel element $eid: $err"
    }
}

# ============================================================================
# RECORDERS
# ============================================================================

section "RECORDERS"

recorder Node -file RoofDisp.out      -time -node 8     -dof 1 disp
recorder Node -file SupportReaction.out -time -node 1 2 -dof 1 reaction
recorder Element -file PierForces.out  -time -ele 1 2 3 4 5 6 basicForce
recorder Element -file SpanForces.out  -time -ele 7 8 9        basicForce
puts "  Recorders defined : OK"

# ============================================================================
# GRAVITY LOADS
# FIX #2 (from review): floor load formula corrected
# Original: -5.0*$g*$rho*$L_span*$T_pier  (≈ 44 100 N — ~12× too large)
# Corrected: -5.0e3*$L_span*$T_pier       (= 3 750 N, i.e. 5 kN/m² floor load)
# If the intent was masonry self-weight use: -$rho*$g*$H_story*$L_span*$T_pier
# ============================================================================

section "GRAVITY LOADS"

set floorLoad [expr -5.0e3 * $L_span * $T_pier]
puts "  Floor load per node : $floorLoad N  (5 kN/m² × L_span × T_pier)"
puts "  Original formula would have given: [expr -5.0*$g*$rho*$L_span*$T_pier] N"

pattern Plain 10 Linear {
    for {set e 1} {$e <= 9} {incr e} {
        eleLoad -ele $e -type -selfWeight 0.0 0.0 [expr -$g]
    }
    load 3 0.0 0.0 $floorLoad 0.0 0.0 0.0
    load 4 0.0 0.0 $floorLoad 0.0 0.0 0.0
    load 5 0.0 0.0 $floorLoad 0.0 0.0 0.0
    load 6 0.0 0.0 $floorLoad 0.0 0.0 0.0
    load 7 0.0 0.0 $floorLoad 0.0 0.0 0.0
    load 8 0.0 0.0 $floorLoad 0.0 0.0 0.0
}

system BandGeneral
numberer Plain
constraints Transformation
test NormUnbalance 100.0 50 5
algorithm Newton
integrator LoadControl 0.01
analysis Static

puts "  Applying gravity loads (100 steps)..."
set ok [analyze 100]
check "Gravity analysis" $ok
```

```
if {$ok != 0} {
    puts "  FATAL — gravity analysis failed. Check material parameters and element setup."
    return
}

loadConst -time 0.0
puts "  Gravity loads held constant. Pseudo-time reset to 0."

# ============================================================================
# FOUNDATION SETTLEMENT
# node 1, DOF 3 (Z) is now free — sp command will work correctly
# ============================================================================

section "FOUNDATION SETTLEMENT"

pattern Plain 20 Linear {
    sp 1 3 -0.001
}

integrator LoadControl 0.01

puts "  Applying settlement (200 steps → 2 mm at node 1)..."
set ok [analyze 200]
check "Settlement analysis" $ok

if {$ok != 0} {
    puts "  WARNING — settlement analysis did not converge cleanly."
    puts "  Continuing with current state, but results may be affected."
}

loadConst -time 0.0
record
puts "  Settlement held constant. Pseudo-time reset to 0."

# ============================================================================
# PUSHOVER LOAD PATTERN
# ============================================================================

section "PUSHOVER SETUP"

pattern Plain 30 Linear {
    load 3 0.20 0.0 0.0 0.0 0.0 0.0
    load 4 0.20 0.0 0.0 0.0 0.0 0.0
    load 5 0.40 0.0 0.0 0.0 0.0 0.0
    load 6 0.40 0.0 0.0 0.0 0.0 0.0
    load 7 0.60 0.0 0.0 0.0 0.0 0.0
    load 8 0.60 0.0 0.0 0.0 0.0 0.0
}
puts "  Triangular load pattern defined (ratios 1:2:3 per floor)"

constraints Transformation
numberer Plain
system BandGeneral
test NormUnbalance 100.0 50 5
algorithm Newton
analysis Static

set targetDisp 0.20
set incr       0.0001
set nSteps     [expr int($targetDisp/$incr)]

integrator DisplacementControl 8 1 $incr

puts "  Target displacement : $targetDisp m"
puts "  Increment           : $incr m"
puts "  Number of steps     : $nSteps"

# ============================================================================
# PUSHOVER ANALYSIS
# FIX #3 (from review): step-by-step loop — no overshoot on fallback
# ============================================================================
```

```
section "PUSHOVER ANALYSIS"

set completedSteps 0
set usingInitial   0

while {$completedSteps < $nSteps} {

    set ok [analyze 1]

    if {$ok == 0} {
        incr completedSteps
        # Revert to standard Newton once we're past a hard spot
        if {$usingInitial} {
            algorithm Newton
            test NormUnbalance 100.0 50 5
            set usingInitial 0
        }
    } else {
        if {!$usingInitial} {
            puts "  Newton failed at step $completedSteps — switching to Newton -initial"
            test NormUnbalance 100.0 200 5
            algorithm Newton -initial
            set usingInitial 1
        }
        set ok [analyze 1]
        if {$ok == 0} {
            incr completedSteps
        } else {
            puts "  STOP — both algorithms failed at step $completedSteps / $nSteps"
            puts "  Roof displacement reached: [expr $completedSteps * $incr] m"
            break
        }
    }

    # Progress report every 500 steps
    if {[expr $completedSteps % 500] == 0 && $completedSteps > 0} {
        puts "  ... step $completedSteps / $nSteps  \
(disp ≈ [expr $completedSteps * $incr] m)"
    }
}

puts ""
puts "  Pushover completed: $completedSteps of $nSteps steps"
puts "  Final roof displacement: [expr $completedSteps * $incr] m"
puts "  Results written to: RoofDisp.out, SupportReaction.out"
puts "  Element results  : PierForces.out, SpanForces.out"
```